```
separate (IDL.SEM_PHASE)¶    --|----------------------------------------------------------------------------------------------¶    --| PRE_FCNS¶    --
|------------------------------------------------------------------------------------------------¶package body PRE_FCNS is¶  use DEF_UTIL;¶  use PRENAME
;¶  use MAKE_NOD;¶  use REQ_UTIL;¶        -- THIS PACKAGE CONTAINS THE PROCEDURE GEN_PREDEFINED_OPERATORS¶        --        WHICH CREATES DEF NODES FOR
 PREDEFINED OPERATORS FOR¬A GIVEN¶        --        TYPE¶        -- NOTE.  GEN_PREDEFINED_OPERATORS CARES WHETHER TYPE IS LIMITED¶        --        OR¬P
RIVATE OR INCOMPLETE; THUS, WHEN PROCESSING TRANSITIVE WITHS,¶        --        POINTERS TO FULL SPECS¬SHOULD BE¬CLEARED AND THEN SET AGAIN¶        --
      WHEN THE FULL SPEC IS SEEN¶¶        -- FIRST-TIME SWITCHES -- GEN_PREDEFINED_OPERATORS IS CALLED¶        --        BEFORE THE STATIC STORAGE IN S
EM_GLOB HAS BEEN INITIALIZED;¶        --        PREDEFINED_BOOLEAN AND¬PREDEFINED_INTEGER ARE SET UP¬HERE¶        --        WHEN THE APPROPRIATE CALL TO
 GEN_PREDEFINED_OPERATORS IS MADE¶  BOOLEAN_IS_INITIALIZED : Boolean := False;¶  INTEGER_IS_INITIALIZED : Boolean := False;¶¶        -- STATIC STORAGE
 -- NODES TO BE REUSED FOR DIFFERENT CALLS¶        --        TO¬GET_PREDEFINED_OPERATORS¶  LEFT_SYMREP       : TREE;¶  RIGHT_SYMREP      : TREE;¶  LEFT_I
NTEGER_IN  : TREE;¶  RIGHT_INTEGER_IN : TREE;¶¶        --================================================================================¶¶        -- INTERNAL¬S
UBPROGRAMS¶¶        -- UTILITY FUNCTIONS TO GENERATE NODES USED BY GEN_PREDEFINED_OPERATORS¶¶        --|----------------------------------------------
--------------------------------------------¶        --|¶  function GEN_IN (SYMREP, TYPE_SPEC : TREE) return TREE is¶  begin¶    return MAKE_IN (AS_SOURC
E_NAME_S =>¬MAKE_SOURCE_NAME_S (LIST => SINGLETON (MAKE_IN_ID¬(LX_SYMREP => SYMREP, SM_OBJ_TYPE => TYPE_SPEC))));¶  end GEN_IN;¶        --|------------
--------------------------------------------------------------------------------¶        --|¶  function GEN_DOUBLE_PARAM (LEFT_IN, RIGHT_IN : TREE) return
 TREE is¶  begin¶    return MAKE_GENERAL_ASSOC_S (LIST => APPEND (SINGLETON (LEFT_IN),¬RIGHT_IN));¶  end GEN_DOUBLE_PARAM;¶        --|------------------
--------------------------------------------------------------------------¶        --|¶  function GEN_SINGLE_PARAM (RIGHT_IN :¬TREE) return TREE is¶  beg
in¶    return MAKE_GENERAL_ASSOC_S (LIST => SINGLETON (RIGHT_IN));¶  end GEN_SINGLE_PARAM;¶        --|------------------------------------------------------
-----------------------------------------¶        --|¶  function GEN_HEADER (RESULT, PARAMS :¬TREE) return TREE is¶  begin¶    return MAKE_FUNCTION_SPEC
¬(AS_NAME => RESULT,¬AS_PARAM_S => PARAMS);¶  end GEN_HEADER;¶        --|--------------------------------------------------------------------------------
------------¶        --|¶  procedure GEN_OP_DEF (OP : OP_CLASS; HEADER : TREE; H : H_TYPE) is¶    DEF :¬TREE;¶  begin¶    DEF := MAKE_DEF_FOR_ID (BLTN_I
D_ARRAY (OP), H);¶    MAKE_DEF_VISIBLE (DEF, HEADER);¶  end GEN_OP_DEF;¶        --|-------------------------------------------------------------------------
---------------------¶        --|¶  function OPS_ARE_NOT_YET_DEFINED (TYPE_SPEC : TREE; OP_FIRST : OP_CLASS; OP_LAST : OP_CLASS) return Boolean¬is¶¬
    -- TESTS IF NAMES FOR OPS IN OP_FIRST .. OP_LAST ARE USED IN¶¬        -- ... THEN COMPILATION¬BUT OPERATIONS NOT YET DEFINED¶¬        -- ... (USED IN F
ULL DECLARATION OF [LIMITED] PRIVATE¬TYPES)¶  begin¶    for OP in OP_FIRST .. OP_LAST loop¶      if BLTN_ID_ARRAY (OP) /= TREE_VOID then¶        decla
re¶¬DEFLIST¬ : SEQ_TYPE := LIST¬(D (LX_SYMREP, BLTN_ID_ARRAY (OP)));¶¬DEF¬ : TREE;¶¬BASE_TYPE¬ : TREE¬  := GET_BASE_TYPE (TYPE_SPEC);¶¬REGION¬ : TREE¬
  := D (XD_REGION, D (XD_SOURCE_NAME, BASE_TYPE));¶¬REGION_DEF : TREE¬  := GET_DEF_FOR_ID¬(REGION);¶        begin¶¬while not¬IS_EMPTY (DEFLIST) loop¶¬
  POP (DEFLIST, DEF);¶¬  if D (XD_SOURCE_NAME, DEF).TY = DN_BLTN_OPERATOR_ID and then D (XD_REGION_DEF, DEF) = REGION_DEF and then GET_BASE_TYPE (D (S
M_OBJ_TYPE, HEAD (LIST¬(D (AS_SOURCE_NAME_S, HEAD (LIST (D (AS_PARAM_S, D (XD_HEADER, DEF)))))))) = BASE_TYPE then¶¬    return False;¶¬  end if;¶¬end
 loop;¶¬return True;¶        end;¶      end¬if;¶    end loop;¶    return False;¶  end OPS_ARE_NOT_YET_DEFINED;¶    --|||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶    --|¶  procedure GEN_PREDEFINED_OPERATORS (TYPE_SPEC :¬TREE; H_IN : H_TYPE) is¶¶
 H : H_TYPE := H_IN;¶¶    BASE_TYPE   : constant TREE := GET_BASE_TYPE (TYPE_SPEC);¶    BASE_STRUCT : constant TREE := GET_BASE_STRUCT (BASE_TYPE);¶¶¬
      -- NODES REUSED FOR SEVERAL CLASSES OF PREDEFINED OPERATOR¶    LEFT_TYPE_IN    : TREE := TREE_VOID;¶    RIGHT_TYPE_IN   : TREE := TREE_VOID;¶    P
ARAMS_TWO     : TREE := TREE_VOID;¶    PARAMS_ONE     : TREE := TREE_VOID;¶    HEADER_BOOLEAN : TREE := TREE_VOID;¶    HEADER_TYPE     : TREE := TREE_V
OID;¶    HEADER_BY_INT  : TREE := TREE_VOID;¶¶¬    ------------------------------------------------------------------------------¶¶¬    -- FUNCTIONS TO GENE
RATE AND OBTAIN REUSED NODES¶¬     -- THESE PERMIT NODES TO BE GENERATED ONLY IF ACTUALLY USED¶        --|------------------------------------------
------------------------------------------¶        --|¶    function USE_LEFT_TYPE_IN¬return TREE is¶    begin¶      if LEFT_TYPE_IN = TREE_VOID t
hen¶        LEFT_TYPE_IN := GEN_IN (LEFT_SYMREP, BASE_TYPE);¶      end¬if;¶      return LEFT_TYPE_IN;¶    end USE_LEFT_TYPE_IN;¶        --|----------
--------------------------------------------------------------------------------¶        --|¶    function USE_RIGHT_TYPE_IN return TREE is¶    begin¶
    if RIGHT_TYPE_IN = TREE_VOID then¶        if RIGHT_SYMREP = LEFT_SYMREP then¶¬RIGHT_TYPE_IN := USE_LEFT_TYPE_IN;¶        else¶¬RIGHT_TYPE_IN := GE
N_IN (RIGHT_SYMREP, BASE_TYPE);¶        end if;¶      end¬if;¶      return RIGHT_TYPE_IN;¶    end USE_RIGHT_TYPE_IN;¶        --|---------------------
--------------------------------------------------------------------¶        --|¶    function USE_PARAMS_TWO return TREE¬is¶    begin¶      if PARAMS_
TWO¬= TREE_VOID then¶        PARAMS_TWO := GEN_DOUBLE_PARAM (USE_LEFT_TYPE_IN, USE_RIGHT_TYPE_IN);¶      end¬if;¶      return PARAMS_TWO;¶    end USE_
PARAMS_TWO;¶        --|-----------------------------------------------------------------------------------¶        --|¶    function USE_PARAMS_
ONE return TREE¬is¶    begin¶      if PARAMS_ONE¬= TREE_VOID then¶        PARAMS_ONE := GEN_SINGLE_PARAM (USE_RIGHT_TYPE_IN);¶      end¬if;¶      retu
rn PARAMS_ONE;¶    end USE_PARAMS_ONE;¶        --|-----------------------------------------------------------------------------------------¶        -
-|¶    function USE_HEADER_BOOLEAN return TREE is¶    begin¶      if HEADER_BOOLEAN = TREE_VOID then¶        HEADER_BOOLEAN := GEN_HEADER (PREDEFINED_
BOOLEAN, USE_PARAMS_TWO);¶      end¬if;¶      return HEADER_BOOLEAN;¶    end USE_HEADER_BOOLEAN;¶        --|------------------------------------------
------------------------------------------¶        --|¶    function USE_HEADER_TYPE return TREE is¶    begin¶      if HEADER_TYPE = TREE_VOID th
en¶        HEADER_TYPE¬:= GEN_HEADER (BASE_TYPE, USE_PARAMS_TWO);¶      end¬if;¶      return HEADER_TYPE;¶    end USE_HEADER_TYPE;¶        --|-------
---------------------------------------------------------------------------¶        --|¶    function USE_HEADER_BY_INT return TREE is¶    begin

¶      if HEADER_BY_INT = TREE_VOID then¶        HEADER_BY_INT := GEN_HEADER (BASE_TYPE, GEN_DOUBLE_PARAM (USE_LEFT_TYPE_IN, RIGHT_INTEGER_IN));¶
 end¬if;¶      return HEADER_BY_INT;¶    end USE_HEADER_BY_INT;¶¶¬      -- PROCEDURES¬TO GENERATE PREDEFINED OPERATORS FOR DIFFERENT CLASSES¶¬        --
¬ OF TYPES¶        --|-------------------------------------------------------------------------------------¶        --|¶    procedure GEN_PREDEF_
EQ is¶    begin¶      for¬OP in CLASS_EQUALITY_OP loop¶        if BLTN_ID_ARRAY (OP)¬/= TREE_VOID then¶¬GEN_OP_DEF (OP, USE_HEADER_BOOLEAN, H);¶
  end if;¶      end¬loop;¶    end GEN_PREDEF_EQ;¶¶¬      -- GENERATES PREDEFINED¬EQUALITY AND INEQUALITY¶¬        --¬ UNLESS THE TYPE IS¬FULL DECLARATIO
N OF¬A PRIVATE¬TYPE,¶¬      --¬ IN WHICH¬CASE EQUALITY AND INEQUALITY HAVE ALREADY BEEN¶¬      --¬ DECLARED¶        --|-----------------------------
--------------------------------------------------------------¶        --|¶    procedure CHECK_PREDEF_EQ¬is¶    begin¶      if OPS_ARE_NOT_YET_DEFINED (B
ASE_TYPE, CLASS_EQUALITY_OP'FIRST,¬CLASS_EQUALITY_OP'LAST) then¶        GEN_PREDEF_EQ;¶      end¬if;¶    end CHECK_PREDEF_EQ;¶        --|------------
-----------------------------------------------------------------------------¶        --|¶    procedure GEN_PREDEF_BOOLEAN is¶    begin¶      for¬OP i
n CLASS_BOOLEAN_OP loop¶        if BLTN_ID_ARRAY (OP)¬/= TREE_VOID then¶¬GEN_OP_DEF (OP, USE_HEADER_TYPE, H);¶        end if;¶      end¬loop;¶      if
 BLTN_ID_ARRAY (OP_NOT) /= TREE_VOID then¶        GEN_OP_DEF (OP_NOT, GEN_HEADER (BASE_TYPE, USE_PARAMS_ONE), H);¶      end¬if;¶    end GEN_PREDEF_BOO
LEAN;¶        --|-------------------------------------------------------------------------------------¶        --|¶    procedure GEN_PREDEF_RELAT
IONAL is¶    begin¶      for¬OP in CLASS_RELATIONAL_OP loop¶        if BLTN_ID_ARRAY (OP)¬/= TREE_VOID then¶¬GEN_OP_DEF (OP, USE_HEADER_BOOLEAN, H);¶
        end if;¶      end¬loop;¶    end GEN_PREDEF_RELATIONAL;¶¶¬      -- GENERATE OPERATORS FOR NUMERIC¬TYPE¶¬      -- LOWER AND UPPER BOUNDS OF REQUI
RED OPERATORS ARE GIVEN,¶¬      --¬ TO ALLOW¬THIS PROCEDURE TO BE USED FOR¬DIFFERENT¬CLASSES¶        --|------------------------------------------
------------------------------------------¶        --|¶    procedure GEN_PREDEF_NUMERIC (FIRST_OP, LAST_OP : OP_CLASS) is¶      HEADER_SINGLE¬: TREE
 :=¬GEN_HEADER (BASE_TYPE, USE_PARAMS_ONE);¶    begin¶      for¬OP in FIRST_OP .. LAST_OP loop¶        if BLTN_ID_ARRAY (OP)¬/= TREE_VOID then¶¬GEN_OP
_DEF (OP, USE_HEADER_TYPE, H);¶        end if;¶      end¬loop;¶      for¬OP in CLASS_UNARY_NUMERIC_OP loop¶        if BLTN_ID_ARRAY (OP)¬/= TREE_VOID
then¶¬GEN_OP_DEF (OP, HEADER_SINGLE, H);¶        end if;¶      end¬loop;¶    end GEN_PREDEF_NUMERIC;¶        --|-----------------------------------
-----------------------------------------------¶        --|¶    procedure GEN_PREDEF_FIXED_MULTIPLY¬is¶    begin¶      if BLTN_ID_ARRAY (OP_MULT)
 /= TREE_VOID then¶        GEN_OP_DEF (OP_MULT, USE_HEADER_BY_INT, H);¶        GEN_OP_DEF (OP_MULT, GEN_HEADER¬(BASE_TYPE, GEN_DOUBLE_PARAM (LEFT_INTE
GER_IN, RIGHT_TYPE_IN)), H);¶      end¬if;¶      if BLTN_ID_ARRAY (OP_DIV) /= TREE_VOID then¶        GEN_OP_DEF (OP_DIV, USE_HEADER_BY_INT, H);¶
end¬if;¶    end GEN_PREDEF_FIXED_MULTIPLY;¶        --|-------------------------------------------------------------------------------------¶
  --|¶    procedure GEN_PREDEF_CAT is¶      COMP_TYPE¬: TREE :=¬D (SM_COMP_TYPE, BASE_TYPE);¶      LEFT_COMP_IN¬: TREE :=¬GEN_IN (LEFT_SYMREP, COMP_T
YPE);¶      RIGHT_COMP_IN¬: TREE :=¬GEN_IN (RIGHT_SYMREP, COMP_TYPE);¶    begin¶      if BLTN_ID_ARRAY (OP_CAT) /= TREE_VOID then¶        GEN_OP_DEF (
OP_CAT, USE_HEADER_TYPE, H);¶        GEN_OP_DEF (OP_CAT, GEN_HEADER (BASE_TYPE, GEN_DOUBLE_PARAM (LEFT_COMP_IN, RIGHT_COMP_IN)),¬H);¶        GEN_OP_DE
F (OP_CAT, GEN_HEADER (BASE_TYPE, GEN_DOUBLE_PARAM (USE_LEFT_TYPE_IN, RIGHT_COMP_IN)), H);¶        GEN_OP_DEF (OP_CAT, GEN_HEADER (BASE_TYPE, GEN_DOUB
LE_PARAM (LEFT_COMP_IN, USE_RIGHT_TYPE_IN)), H);¶      end¬if;¶    end GEN_PREDEF_CAT;¶        --|------------------------------------------------------
--------------------------------¶        --|¶    procedure GEN_PREDEF_EXP is¶    begin¶      if BLTN_ID_ARRAY (OP_EXP) /= TREE_VOID then¶
 GEN_OP_DEF (OP_EXP, USE_HEADER_BY_INT, H);¶      end¬if;¶    end GEN_PREDEF_EXP;¶        --|------------------------------------------------------------
-------------------------------¶        --|¶    procedure GEN_PREDEF_ARRAY is¶      COMP_TYPE : TREE := GET_BASE_TYPE¬(D (SM_COMP_TYPE, BASE_STRUCT)
);¶    begin¶¬¬    -- CHECK THAT COMPONENT TYPE EXISTS¬(1.E. NOT¬PRIOR ERROR)¶      if COMP_TYPE = TREE_VOID then¶        return;¶      end¬if;¶¶¬¬
 -- IF¬IT IS A ONE-DIMENSIONAL ARRAY¶      if IS_EMPTY (TAIL (LIST¬(D (SM_INDEX_S, BASE_STRUCT)))) then¶¶¬¬¬  -- GENERATE CONCATENATION OPERATORS¶
    if OPS_ARE_NOT_YET_DEFINED (BASE_TYPE, OP_CAT, OP_CAT) then¶¬GEN_PREDEF_CAT;¶        end if;¶      end¬if;¶¶¬¬    -- FOR AN ARRAY¬WITH PRIVATE COM
PONENTS¶      if H.IS_IN_SPEC and then IS_PRIVATE_TYPE (COMP_TYPE) then¶¶¬¬¬  -- RELATIONAL AND¬BOOLEAN OPERATORS NOT DEFINED¬YET¶        return;¶
  end¬if;¶¶      if IS_EMPTY (TAIL (LIST¬(D (SM_INDEX_S, BASE_STRUCT)))) then¶        if OPS_ARE_NOT_YET_DEFINED (BASE_TYPE, CLASS_RELATIONAL_OP'FIRST
, CLASS_RELATIONAL_OP'LAST)¬then¶¬GEN_PREDEF_RELATIONAL;¶        end if;¶¶      end¬if;¶      if IS_BOOLEAN_TYPE (COMP_TYPE) then¶        if OPS_ARE_N
OT_YET_DEFINED (BASE_TYPE, CLASS_BOOLEAN_OP'FIRST, CLASS_BOOLEAN_OP'LAST) then¶¬GEN_PREDEF_BOOLEAN;¶        end if;¶      end¬if;¶    end GEN_PREDEF_A
RRAY;¶        --|-------------------------------------------------------------------------------------¶        --|¶    procedure GEN_PREDEF_UNIV_
REAL is¶      UI_TYPE       : TREE := MAKE (DN_UNIVERSAL_INTEGER);¶      LEFT_UI_IN   : TREE;¶      RIGHT_UI_IN  : TREE;¶      HEADER_UI_UR : TREE;¶
   HEADER_UR_UI : TREE;¶    begin¶      if BLTN_ID_ARRAY (OP_MULT) /= TREE_VOID or BLTN_ID_ARRAY (OP_DIV) /= TREE_VOID then¶        RIGHT_UI_IN¬ := GE
N_IN (RIGHT_SYMREP, UI_TYPE);¶        HEADER_UR_UI := GEN_HEADER (BASE_TYPE, GEN_DOUBLE_PARAM (USE_LEFT_TYPE_IN, RIGHT_UI_IN));¶      end¬if;¶      if
 BLTN_ID_ARRAY (OP_DIV) /= TREE_VOID then¶        GEN_OP_DEF (OP_DIV, HEADER_UR_UI, H);¶      end¬if;¶      if BLTN_ID_ARRAY (OP_MULT) /= TREE_VOID th
en¶        LEFT_UI_IN¬ := GEN_IN (LEFT_SYMREP, UI_TYPE);¶        HEADER_UI_UR := GEN_HEADER (BASE_TYPE, GEN_DOUBLE_PARAM (LEFT_UI_IN, USE_RIGHT_TYPE_I
N));¶        GEN_OP_DEF (OP_MULT, HEADER_UI_UR, H);¶        GEN_OP_DEF (OP_MULT, HEADER_UR_UI, H);¶      end¬if;¶    end GEN_PREDEF_UNIV_REAL;¶
  --|---------------------------------------------------------------------------------¶        --|¶    procedure GEN_PREDEF_UNIV_FIXED is¶
begin¶      if BLTN_ID_ARRAY (OP_MULT) /= TREE_VOID then¶        GEN_OP_DEF (OP_MULT, USE_HEADER_TYPE, H);¶      end¬if;¶      if BLTN_ID_ARRAY (OP_DI
V) /= TREE_VOID then¶        GEN_OP_DEF (OP_DIV, USE_HEADER_TYPE, H);¶      end¬if;¶    end GEN_PREDEF_UNIV_FIXED;¶¶  begin -- GEN_PREDEFINED_OPERATOR
S¶    if BASE_TYPE = TREE_VOID or else IS_LIMITED_TYPE (BASE_TYPE) then¶      return;¶    end if;¶¶    case CLASS_TYPE_SPEC'(BASE_STRUCT.TY) is¶
when DN_L_PRIVATE | DN_TASK_SPEC | DN_INCOMPLETE | CLASS_CONSTRAINED =>¶        Put_Line ("!! GEN_PREDEFINED_OPERATORS: IMPOSSIBLE TYPE");¶        rai

se Program_Error;¶¶      when DN_PRIVATE =>¶        CHECK_PREDEF_EQ;¶      when DN_RECORD | DN_ACCESS =>¶        CHECK_PREDEF_EQ;¶      when DN_ENUMER
ATION =>¶        if not BOOLEAN_IS_INITIALIZED then¶¬PREDEFINED_BOOLEAN¬   := TYPE_SPEC;¶¬LEFT_SYMREP¬   := FIND_SYM ("LEFT");¶¬RIGHT_SYMREP¬   := FIN
D_SYM ("RIGHT");¶¬BOOLEAN_IS_INITIALIZED := True;¶        end if;¶        CHECK_PREDEF_EQ;¶        GEN_PREDEF_RELATIONAL;¶        if IS_BOOLEAN_TYPE (
BASE_TYPE) then¶¬GEN_PREDEF_BOOLEAN;¶        end if;¶      when DN_INTEGER =>¶        if not INTEGER_IS_INITIALIZED then¶¬PREDEFINED_INTEGER¬   := BAS
E_TYPE;¶¬LEFT_INTEGER_IN¬   := GEN_IN (LEFT_SYMREP, PREDEFINED_INTEGER);¶¬RIGHT_INTEGER_IN¬   := GEN_IN (RIGHT_SYMREP, PREDEFINED_INTEGER);¶¬INTEGER_I
S_INITIALIZED := True;¶        end if;¶        CHECK_PREDEF_EQ;¶        GEN_PREDEF_RELATIONAL;¶        GEN_PREDEF_NUMERIC (CLASS_INTEGER_OP'FIRST, CLA
SS_INTEGER_OP'LAST);¶        GEN_PREDEF_EXP;¶      when DN_FLOAT¬=>¶        CHECK_PREDEF_EQ;¶        GEN_PREDEF_RELATIONAL;¶        GEN_PREDEF_NUMERIC
 (CLASS_FLOAT_OP'FIRST,¬CLASS_FLOAT_OP'LAST);¶        GEN_PREDEF_EXP;¶      when DN_FIXED¬=>¶        CHECK_PREDEF_EQ;¶        GEN_PREDEF_RELATIONAL;¶
        GEN_PREDEF_NUMERIC (CLASS_FIXED_OP'FIRST,¬CLASS_FIXED_OP'LAST);¶        GEN_PREDEF_FIXED_MULTIPLY;¶      when DN_UNIVERSAL_INTEGER =>¶        G
EN_PREDEF_EQ;¶        GEN_PREDEF_RELATIONAL;¶        GEN_PREDEF_NUMERIC (CLASS_INTEGER_OP'FIRST, CLASS_INTEGER_OP'LAST);¶        GEN_PREDEF_EXP;¶
 when DN_UNIVERSAL_REAL =>¶        GEN_PREDEF_EQ;¶        GEN_PREDEF_RELATIONAL;¶        GEN_PREDEF_NUMERIC (CLASS_FLOAT_OP'FIRST,¬CLASS_FLOAT_OP'LAST
);¶        GEN_PREDEF_EXP;¶        GEN_PREDEF_UNIV_REAL;¶      when DN_UNIVERSAL_FIXED¬=>¶        GEN_PREDEF_UNIV_FIXED;¶      when DN_ARRAY¬=>¶
  CHECK_PREDEF_EQ;¶        GEN_PREDEF_ARRAY;¶    end case;¶  end GEN_PREDEFINED_OPERATORS;¶¶    --|---------------------------------------------------
----------------------------------------------¶end PRE_FCNS;¶
```